{PYTHON}

COMPLETE NOTES

Code Academy • SEA Technologies Media

Complete Python Course

Learn. Build. Innovate.

Created by Emmanuel Adedayo

TABLE OF CONTENTS

01 The Print Function ..... Page 1

02 Arithmetic & Comparison Operators ..... Page 2

03 User Input ..... Page 3

04 Project 1 — Simple Calculator ..... Page 4

05 Lists & Tuples ..... Page 5

06 Loops ..... Page 6

07 Functions ..... Page 7

08 Dictionaries ..... Page 8

09 Data Types ..... Page 9

10 Conditional Statements ..... Page 10

11 Project 2 — Grade Calculator ..... Page 11

1 The Print Function

Topic 1

Introduction to Python

The `print()` function is the very first thing you learn in Python. It displays text (output) on the screen.

1.1 What is `print()`?

The `print()` function sends text to the screen so you can see it. Whatever you put inside the quotation marks will be displayed.

Key Point: The `print()` function is used to **output** information to the screen. It is one of the most commonly used functions in Python.

1.2 Syntax

```
print("Your text here")
```

1.3 How It Works

- `print("Hello World")` — Prints the text **Hello World**
- `print("Welcome to Code Academy")` — Prints the text **Welcome to Code Academy**
- `print("My name is Emmanuel Adedayo")` — Prints your name
- `print("Starting with the first thing 'PRINT'")` — You can include single quotes inside double quotes

Tip: Each `print()` statement creates a **new line** by default. So every `print()` you write will appear on its own line.

1.4 Rules to Remember

1. Text inside `print()` must be wrapped in **quotation marks** — either `" "` or `' '`
2. Without quotes, Python thinks you are referring to a variable, not text
3. Python is **case-sensitive** — `Print` and `print` are different
4. Each `print()` creates a new line of output

2 Arithmetic & Comparison Operators

Topic 2 Operators in Python

Operators are special symbols that perform operations on variables and values.

2.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Symbol	Example
Addition	`+`	`a + b`
Subtraction	`-`	`a - b`
Multiplication	`*`	`a * b`
Division	`/`	`a / b`
Floor Division	`//`	`a // b`
Power	`**`	`a ** b`
Modulus	`%`	`a % b`

Remember: With `a = 20` and `b = 6`:
`a / b = 3.333...` (regular division) vs `a // b = 3` (floor division removes the decimal)
`a ** b = 64000000` (20 to the power of 6)
`a % b = 2` (remainder when 20 is divided by 6)

2.2 Comparison Operators

Comparison operators compare two values and return `True` or `False`.

Operator	Symbol	Example	Result
Greater Than	`>`	`20 > 6`	`True`
Less Than	`<`	`20 < 6`	`False`
Equal To	`==`	`20 == 6`	`False`
Not Equal To	`!=`	`20 != 6`	`True`
Greater or Equal	`>=`	`20 >= 6`	`True`
Less or Equal	`<=`	`20 <= 6`	`False`

Tip: Comparison operators always return a **Boolean** value — either `True` or `False`. These are very useful in conditional statements (if/else).

3 User Input

Topic 3

Getting Input from Users

The `input()` function allows your program to receive data from the user while it is running.

3.1 What is `input()`?

The `input()` function pauses the program, waits for the user to type something, and then **returns** what they typed as a **string**.

```
variable = input("Prompt message here")
```

3.2 How It Works

- `name = input("What is your name ")` — Asks the user for their name and stores it in the variable `name`
- `course = input("Which course do you like...")` — Asks the user for their favourite course
- `age = input("How old are you...")` — Asks the user for their age

3.3 Combining Input with Print

You can use the `+` operator to join (concatenate) strings together:

```
print("Hello " + name + " You're welcome...")
```

Key Point: The `+` operator joins strings together. This is called **string concatenation**. Each piece must be a string — if you want to join a number, you must convert it first using `str()`.

3.4 Important Rules

1. `input()` **always returns a string**, even if the user types a number
2. To use input as a number, convert it: `int(input(...))` or `float(input(...))`
3. You can concatenate strings with the `+` operator
4. The prompt message inside `input()` is shown to the user before they type

Warning: If you try to do math with an `input()` result without converting it, Python will give an error! Always convert to `int()` or `float()` when you need numbers.

4 Project 1 — Simple Calculator

PROJECT Hands-On Project

Simple Calculator

A beginner project that takes two numbers from the user and performs basic arithmetic operations (addition, subtraction, multiplication, division).

4.1 Concepts Used

This project combines what you have learned so far:

- `float(input())` — Getting number input from the user
- Arithmetic operators: `+` `-` `*` `/`
- `print()` — Displaying the results
- String concatenation with `+`

4.2 How It Works Step by Step

1. **Get numbers:** Use `float(input())` to get two numbers from the user
2. **Calculate:** Perform addition, subtraction, multiplication, and division
3. **Display:** Use `print()` to show each result with a label

4.3 Key Notes

Why `float()`?
The `float()` function converts the user's input into a decimal number. This allows the calculator to handle both whole numbers and decimals (like `5.5`).

Note: In the source code, `int()` is used to convert the result back to integer before printing. This works for whole number results, but for division it may cause issues if the result has decimals.

5 Lists & Tuples

Topic 4

Data Collections

Lists and Tuples are used to store **multiple items** in a single variable.

5.1 Lists

A list is an **ordered, changeable** collection of items, written with **square brackets** `[]`.

```
students = ["Emmanuel", "Dave", "John", "Jane", "Mike"]
courses = ["HTML", "CSS", "JavaScript", "Python", "Git", "Django"]
scores = [85, 92, 78, 65, 90]
```

5.1.1 Accessing Items

Each item has a position number called an **index**, starting from `0`:

- `students[0]` returns **"Emmanuel"** (first item)
- `courses[1]` returns **"CSS"** (second item)
- `scores[-1]` returns **90** (last item — negative index counts from the end)

Key Point: Python indexing starts at `0`, not `1`. So the first item is at index `0`, the second at `1`, and so on. Negative indices count from the end: `-1` is the last item.

5.1.2 Useful Functions

`len()` returns the number of items in a list:

```
print(len(students))  # Output: 5
```

5.1.3 Updating an Item

You can change a specific item by assigning a new value to its index:

```
students[0] = "Emmanuel Adedayo"
```

This replaces the first item with **"Emmanuel Adedayo"**.

5.2 Tuples

A tuple is an **ordered, unchangeable** collection, written with **parentheses** `()`.

```
days = ("Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday")
```

Feature	List	Tuple
Brackets	`[]`	`()`
Changeable?	Yes	No
Ordered?	Yes	Yes

6 Loops

Topic 5

For Loop, While Loop & Multiplication Table

Loops allow you to **repeat** a block of code multiple times without rewriting it.

6.1 For Loop

A `for` loop iterates over a sequence (like a range of numbers or a list).

6.1.1 Using `range()`

```
for i in range(5):
    print(i)       # Prints 0, 1, 2, 3, 4
```

Key Point: `range(5)` generates numbers from `0` to `4` (it stops **before** 5). `range(1, 12)` generates numbers from `1` to `11`.

6.1.2 Looping Through a Collection

You can also loop through items in a list or tuple:

```
courses = ("HTML", "CSS", "JAVASCRIPT", "PYTHON")
for courses in courses:
    print("I am learning " + courses)
```

6.2 While Loop

A `while` loop repeats **as long as** a condition is `True`.

```
count = 1
while count <= 100:
    print("Count is: " + str(count))
    count = count + 1
```

Warning: Always make sure the condition will eventually become `False`, otherwise you create an **infinite loop** that never stops! In the code above, `count = count + 1` ensures the loop ends when count reaches 101.

6.3 Real World Example — Multiplication Table

This combines `input()`, `int()`, a `for` loop, and string concatenation to build a working multiplication table:

```
number = int(input("Enter a number: "))
for i in range(1, 11):
    print(str(number) + " x " + str(i) + " = " + str(number * i))
```

Feature	For Loop	While Loop
Used when	You know how many times	You don’t know how many times
Controlled by	`range()` or collection	A condition
Risk	Low	Infinite loop if condition never becomes False

7 Functions

Topic 6

Defining & Calling Functions

A function is a **reusable block of code** that performs a specific task. You define it once and call it as many times as you want.

7.1 Basic Function

A basic function has no parameters — it does the same thing every time:

```
def greet():
    print("Hello, welcome to Code Academy")

greet()    # Call it!
```

Key Point: The `def` keyword defines a function. The function name is followed by `()` and a colon `:`. The code inside must be **indented**.

7.2 Function with Parameters

You can pass data into a function using **parameters**:

```
def greet_students(name):
    print("Hello " + name + " Welcome to python!")

greet_students("Emmanuel")
```

The parameter `name` receives the value you pass when calling the function. You can call it with different values each time.

7.3 Function with Return Value

A function can **calculate** something and **return** the result using the `return` keyword:

```
def calculate_area(length, width):
    area = length * width
    return area

room1 = calculate_area(10, 5)  # Returns 50
room2 = calculate_area(8, 6)   # Returns 48
```

Tip: The `return` statement sends a value **back** to wherever the function was called. You can then store that value in a variable (like `room1`) or use it in other calculations.

8 Dictionaries

Topic 7

Key-Value Pairs

A dictionary stores data as **key-value pairs**, written with **curly braces** `{}`.

8.1 Creating a Dictionary

Each piece of data has a **key** (the label) and a **value** (the actual data):

```
students = {
    "name": "Emmanuel Adedayo",
    "age": 20,
    "course": "python",
    "email": "seatechnologiesmedia@gmail.com",
    "is_active": True
}
```

8.2 Accessing Values

Use the **key** in square brackets to get the value:

- `students["name"]` returns **"Emmanuel Adedayo"**
- `students["age"]` returns **20**

8.3 Checking If a Key Exists

Before accessing a key, you can check if it exists using the `in` keyword:

```
if "email" in students:
    print("email found " + students["email"])
```

8.4 Dictionary of Dictionaries

You can nest dictionaries inside dictionaries to store complex data:

```
students = {
    "student1": {"name": "Emmanuel", "score": 85},
    "student2": {"name": "Dave", "score": 72},
    "student3": {"name": "John", "score": 55}
}
```

Access nested data using `.items()`:

```
for student_id, info in students.items():
    print(info["name"] + " - " + info["grade"])
```

Data Structure	Best For
List `[]`	Ordered items, can change
Tuple `()`	Ordered items, cannot change
Dictionary `{}`	Labelled data (key-value)

9 Data Types

Topic 8

String, Integer, Float, Boolean

Every value in Python has a **data type** — it tells Python what kind of data it is.

9.1 The Four Basic Data Types

Data Type	Keyword	Example	Description
String	`str`	`"Emmanuel"`	Text, wrapped in quotes
Integer	`int`	`20`	Whole numbers
Float	`float`	`9.99`	Decimal numbers
Boolean	`bool`	`True/False`	True or False values

9.2 Examples from the Code

```
# String
name = "Emmanuel"

# Integer
age = 20

# Float
price = 9.99

# Boolean
is_free = True
```

9.3 Checking Data Types with `type()`

The `type()` function tells you the data type of any variable:

```
print(type(name))    # <class 'str'>
print(type(age))     # <class 'int'>
print(type(price))   # <class 'float'>
print(type(is_free)) # <class 'bool'>
```

Key Point: Knowing the data type is important because different types behave differently. For example, you can add two numbers (`20 + 5 = 25`), but you cannot add a string and a number without converting first.

Tip: Use `str()` to convert to string, `int()` to convert to integer, `float()` to convert to decimal, and `bool()` to convert to boolean.

Code Academy • SEA Technologies Media • Learn. Build. Innovate.

10 Conditional Statements

Topic 9

If, Elif, Else

Conditional statements allow your program to make **decisions** — different code runs depending on whether a condition is `True` or `False`.

10.1 Basic If / Else

```
if condition:
    # runs if condition is True
else:
    # runs if condition is False
```

Example from the code:

```
years = 18
if years >= 18:
    print("Congratulation You are an Adult")
else:
    print("Sorry You are not an Adult yet")
```

10.2 If / Elif / Else

When you have **multiple conditions**, use `elif` (else if) to check them one by one:

```
if score >= 70:
    print("Distinction")
elif score >= 50:
    print("Pass")
elif score >= 40:
    print("Nearly Pass")
else:
    print("Fail")
```

Key Point: Python checks conditions from **top to bottom**. Once it finds a `True` condition, it runs that block and **skips the rest**. So the order matters!

10.3 With User Input

You can combine `input()` with conditions to make interactive programs:

```
score = float(input("Enter your score: "))
if score >= 70:
    print("Excellent! You got a Distinction")
# ... more conditions
```

10.4 Login System

A real-world example using `==` to check if the password matches:

```
password = input("Enter password: ")
if password == "python123":
    print("Access granted! Welcome")
else:
    print("Wrong password. Access denied")
```

Keyword	Purpose
`if`	First condition to check
`elif`	Additional condition (checked only if previous were False)
`else`	Runs if all conditions above were False

11 Project 2 — Grade Calculator

PROJECT Hands-On Project

Grade Calculator

A real-world project that calculates student grades, displays formatted results, and saves them to a file. This project combines functions, loops, conditionals, input, and file handling.

11.1 Concepts Used

This project brings together **everything** you have learned:

- **Functions** — `get_grade()`, `get_remark()`, `save_result()`, `main()`
- **Conditional Statements** — `if/elif/else` for grade classification
- **Loops** — `for` loop to collect scores and display results
- **Lists** — storing subjects and scores
- **Input** — getting student name and scores
- **File Handling** — writing results to `results.txt`
- **Error Handling** — `try/except` for invalid input
- **f-strings** — formatted output

11.2 How It Works Step by Step

1. **Run main():** The program starts and displays a welcome banner
2. **Get name:** Asks for the student's full name
3. **Define subjects:** Creates a list of 5 subjects
4. **Collect scores:** Uses a `for` loop with `try/except` to safely get each score (0-100)
5. **Calculate:** Computes total, average, grade, and remark
6. **Display:** Prints a formatted result card
7. **Save:** Writes the results to `results.txt`

11.3 Key Functions Explained

Function	Purpose
`get_grade(average)`	Returns DISTINCTION, PASS, NEARLY PASS or FAIL
`get_remark(average)`	Returns a motivational remark based on grade
`save_result(...)`	Writes student result to a text file
`main()`	Runs the entire program

11.4 Grade Classification

Average Score	Grade
70 and above	DISTINCTION
50 — 69	PASS
40 — 49	NEARLY PASS
Below 40	FAIL

Tip: This project shows how real applications work — they combine multiple concepts (input, functions, loops, conditions, file handling) to build something useful. Try modifying it! Add more subjects, change the grade thresholds, or add a GPA calculation.